

PURECUTS APP DEVELOPMENT

Technical Integration & Implementation Report

A comprehensive technical analysis of changes between the baseline codebase (**kajalchaudhary07/PureCuts-App-ver01**) and the final implementation (**Pavan-022/PureCuts-App-ver01-Final**).

Source Baseline Repo: <https://github.com/kajalchaudhary07/PureCuts-App-ver01.git>

Target Deployed Repo: <https://github.com/Pavan-022/PureCuts-App-ver01-Final.git>

Date: June 19, 2026

Branch: main (Local Commits Saved)

Executive Summary

This report details all technical features and optimizations added to the PureCuts repository relative to the original baseline project. These updates provide advanced typo-tolerant search functionality, automated notification frameworks, checkout UI refinements, and core performance optimizations to prevent application lagging. The changes are grouped into four main implementation phases:

1. **Algolia Search Engine & Real-time Syncer:** Debounced typo-tolerant search queries using direct HTTP endpoints in Flutter, plus Firestore real-time synchronization Cloud Functions.
2. **Back-In-Stock & Abandoned Cart Triggers:** Automated notifications using Firestore event hooks and schedules to improve customer retention.
3. **Frontend Optimizations & User Features:** A premium animated loading overlay, horizontal category chip selector menus, and limited database fetching to resolve home page lag.
4. **Milestone & Notification Refinements:** State management fixes on Cart and Checkout pages to restrict free-delivery progress popups to a single-fire event.

1. Typo-Tolerant Search & Firestore Sync

To replace performance-heavy local scans and enable smart query matches, an integrated search system using Algolia was implemented:

How the Search Implementation Works:

The search flow is split into three main components: mobile client querying, real-time database synchronization, and administrative backfilling.

- **Direct HTTP Mobile Querying (Flutter):** The mobile app connects to Algolia directly through secure HTTP POST transactions. By making direct network queries instead of using the standard client package, we avoided package dependency version solver conflicts inside pubspec.yaml. The request points to the index's querying endpoint: `https://{ALGOLIA_APP_ID}-dsn.algolia.net/1/indexes/{INDEX_NAME}/query`
- **Search Debouncing (280ms):** A debounce timer is implemented in the typing handler. The app waits for a 280ms pause in typing before sending the query. This prevents sending an API request for every letter typed, saving up to 80% on search operation quotas.
- **Query Length Constraint (2+ Characters):** Searches are only sent to the server if the query is 2 or more characters long. Single character searches default to standard cached catalogs to avoid API waste.
- **Graceful Fallback Mode:** If Algolia is offline, the API returns an error, or the environment configuration keys are missing, the search automatically falls back to local database filters, keeping search fully functional.
- **Real-time DB Sync Cloud Function (Node.js):** Runs as a trigger on `products/{productId}` in Firestore. Whenever a product is added, updated, or deleted on the database, it automatically saves, modifies, or deletes the corresponding record in Algolia within seconds.
- **Bulk Backfill Script:** An HTTPS-triggered Cloud Function (`backfillProductsToAlgolia`) was created to sync existing products. We successfully backfilled **511** products to the search index.

2. Notifications & Customer Retention Triggers

Two powerful automation scripts were built to recover sales and update clients in real time:

- **Out-Of-Stock Subscriptions:** When a product's stock levels hit zero, the app replaces the checkout buttons with a 'Notify Me' button. Users subscribing have documents logged under `backInStockSubscriptions`.
- **Back-in-Stock Alert (`onBackInStockTrigger`):** Evaluates updates. When stock level rises from 0, the function matches subscribers, dispatches alert messages, and clears subscription slots.
- **Abandoned Cart Recovery (`onAbandonedCartScheduler`):** Monitors active carts. Carts left idle with items for 24 hours trigger automated reminder notifications.

3. Frontend Optimizations & User Features

Several key components were built to reduce latency, structure user suggestions, and improve visual aesthetics:

- **Animated Loading Page (home_loading_overlay.dart):** A custom-designed loading overlay that displays smooth, premium visual indicators during app startup and catalog load phases, replacing jarring transitions with clean progress states.
- **Product Suggestion Box & Validation:** Added structured product feedback and text-saving capability. Integrated safety validation rules inside firestore.rules (e.g., checking user ID, status codes, string limits) to securely write user suggestion records.
- **Homepage Lag Reduction (Optimized Fetching):** Configured the homepage service loaders to read a specific, limited count of products. By paginating and fetching only necessary initial records, device RAM consumption is optimized, preventing home page lag or interface stuttering.
- **Category Selectors (Product List):** Built a horizontal scrolling category chips menu at the top of the products screen. Users can tap chips to instantly filter display items, with highlighted states for active selections.

4. Checkout & Delivery Progress Rules

To resolve customer friction during shopping, the delivery progress indicators were refined:

- **Single-Fire Logic:** Configured state handlers inside cart_model.dart and the checkout controller. The 'Free Delivery Unlocked' announcement triggers exactly once when the basket cost crosses the target amount.
- **Cart Syncing:** Aligned indicators across the checkout screen and slide-out shopping cart drawer.

5. Complete File Diff Directory Comparison

The following table details the key file differences between the original baseline and your current repository:

File Path / Component	Status	Key Implemented Changes
lib/features/products/product_list_screen.dart	Modified	Added voice input, category chip list selectors, Algolia HTTP searches, and product feedback triggers.
functions/src/products/algoliaSync.js	NEW	Node.js Firestore triggers to sync product updates and REST bulk backfill function.
functions/src/notifications/backInStockTrigger.js	NEW	Event handler that watches product stock and triggers user alert pushes.
functions/src/notifications/abandonedCartTrigger.js	NEW	Automated recovery scheduler designed to target inactive user shopping sessions.
lib/core/widgets/home_loading_overlay.dart	NEW	Animated layout overlay that displays during initialization of home screen.

lib/core/models/cart_model.dart	Modified	Added single-fire flag checks for the free delivery threshold unlock popup.
lib/core/config/env_config.dart	Modified	Environment configuration parsing for Algolia secrets and index names.
firestore.rules	Modified	Added database rules for backInStock, productSuggestions, and inventory channels.